PRAKTIKUM 1
PENGENALAN MODUL LYNXMOTION

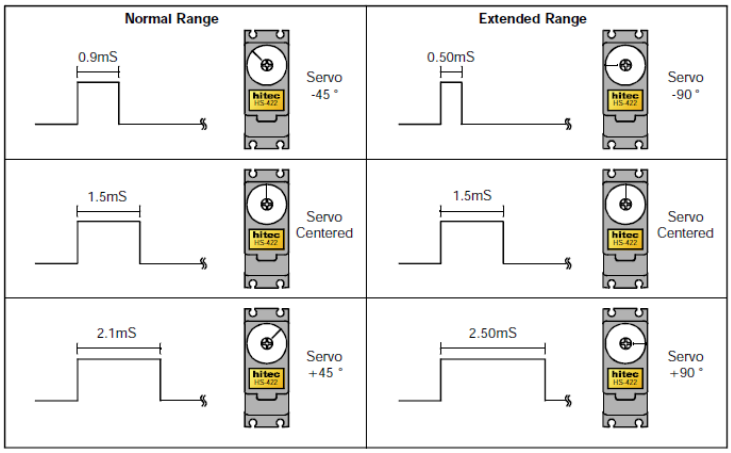
Pendahuluan

➢ Motor Servo

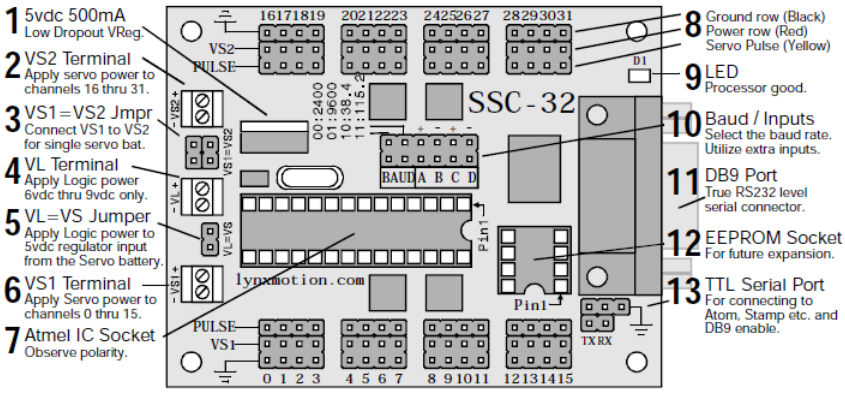
Motor servo awalnya digunakan pada perangkat *radio controlled* (R/C) seperti mobil, perahu dan pesawat. Motor servo mampu memberikan pengaturan yang presisi terhadap mekanisme *steering*, kemudi dan *throttle* menggunakan sinyal yang mudah untuk di-transmit dan diterima. Sinyal tersebut adalah *positive going pulses* dengan rentang antara 0.9 ms sampai 2.1 ms dan diulang sebanyak 50 kali dalam satu detik (setiap 20ms). Posisi yang dihasilkan oleh *output shaft* sebanding dengan lebar dari pulsa seperti pada gambar berikut dibawah ini.

Pada aplikasinya dalam *radio-controlled*, servo dibatasi untuk tidak berputar melebihi 90°. sehingga pulsa dengan lebar 0.9 ms sampai 2.1 ms akan menyebabkan servo berputar dalam rentang 90°. Pada umumnya servo memiliki kemampuan berputar lebih dari 90^0 berdasarkan *mechanical range*-nya. Kebanyakan servo mampu berputar hingga 180° bahkan lebih.

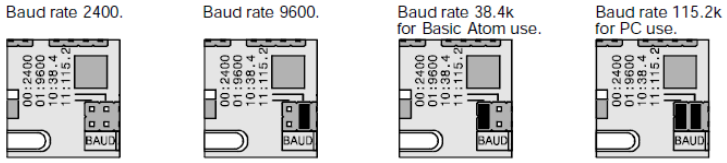
Pada modul SSC-32 dapat digunakan pada *extra range* dengan unit nilai sebesar 500 untuk pulsa dengan lebar 0.5 ms dan unit nilai sebesar 2500 untuk pulsa dengan lebar 2.5 ms. Perubahan unit nilai sebesar 1 akan menghasilkan perubahan lebar pulsa sebesar 1 us. Resolusi dari perubahan sudut adalah 0.09^o/unit nilai (180°/2000).



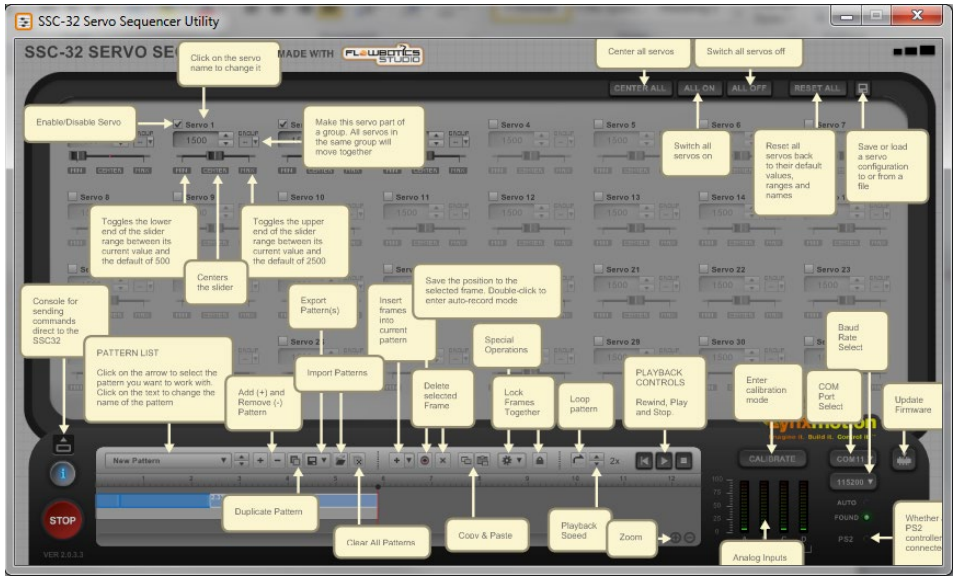
- Konfigurasi *Hardware* SSC-32 Lynxmotion



- Konfigurasi *Baud Rate*



- *Software SSC-32 Servo Sequencer Utility*



Tujuan

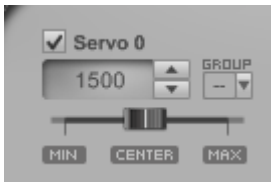
1. Mahasiswa dapat memahami prinsip kerja dari motor servo
2. Mahasiswa dapat memahami cara mengoperasikan modul SSC-32 Lynxmotion
3. Mahasiswa dapat memahami cara mengoperasikan servo dengan modul robot lengan AL5A.

Alat dan Bahan

1. Robot Lengan AL5A
2. SSC-32 Lynxmotion
3. Software SSC-32 Servo Sequencer Utility
4. USB to Serial
5. *Power Suply*
6. Busur dan penggaris

Prosedur Percobaan

1. *Servo Move*
   a. Pasang konektor satu buah motor servo pada modul SSC-32 Lynxmotion dengan ID 0.
   b. Nyalakan Modul SSC-32 Lynxmotion dengan menyambungkan pada *power supply* dengan tegangan 5V – 6V.
   c. Sambungkan modul SSC-32 Lynxmotion dan komputer dengan menekan *button* Connect dan pilih Port COM yang aktif.
   d. Pada aplikasi SSC-32 Servo Sequencer aktifkan ID servo 0 dengan memberikan tanda (√).



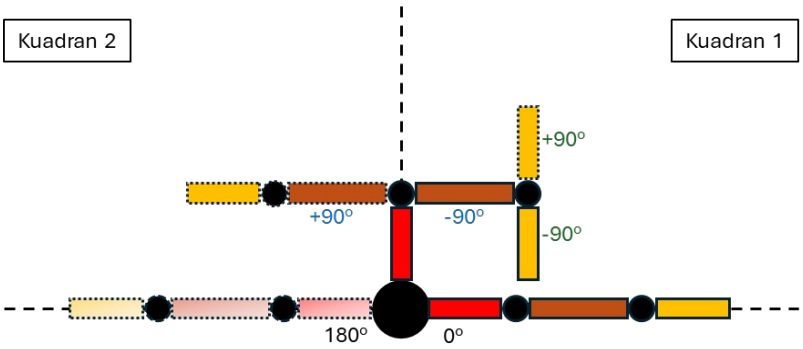
   e. Atur nilai servo dengan cara menggeser posisi *slider*, **amati hasilnya** dan **ukur sudut** yang dihasilkan motor servo dengan busur.
   f. **Catat hasil** pengamatan anda sesuai **tabel 1**.
   g. Untuk **servo 1** catat nilai periode saat **lengan** pada **posisi rata air** (horizontal).
   h. Terdapat tiga tombol dalam bentuk kata yaitu “MIN”, ”MAX” dan “CENTER”, **jelaskan** masing-masing fungsi dari tombol tersebut.

2. *Group Move*
   a. Hubungkan dua motor servo pada modul SSC-32 dengan ID 0 dan ID 1.
   b. Nyalakan Modul SSC-32 Lynxmotion dengan menyambungkan pada *power supply* dengan tegangan 5V – 6V.
   c. Pada Aplikasi SSC-32 Servo Sequencer aktifkan ID servo 0 dan ID servo 1 dengan memberikan tanda (√).
   d. Pada menu list “GROUP” berikan nilai 1 untuk ID servo 0 dan untuk ID servo 1.

e. Atur nilai servo dengan cara menggeser posisi *slider*, **amati hasilnya** dan **jelaskan**.

Data Hasil Pengamatan

Gambar berikut adalah acuan untuk melakukan pengukuran sudut dari mekanik lengan robot. Tulisan warna hitam untuk sudut servo 0, warna biru untuk sudut servo 2 dan warna hijau untuk sudut servo 3.



Berikut adalah tabel yang berisi data pengukuran. Lebar pulsa diisi dengan nilai dari posisi slider dalam satuan us. Sudut servo didapatkan dari pengukuran dengan alat ukur (busur) antara mekanik robot dengan meja kerja (gunakan milimeter blok untuk mempermudah pengukuran).

Tabel 1 Data pengukuran sudut servo dengan masukan berupa lebar pulsa

No.	Lebar Pulsa	Sudut Servo 0 (°)	Sudut Servo 2 (°)	Sudut Servo 3 (°)
		0°-180°	*-90° - +90°*	*-90° - +90°*
1	500			
2				
3				
4				
5	1500			
6				
7				
8				
9				
10	2500			

Tabel 2 Data Pengukuran Servo

No.	Servo Nomor	Lebar Pulsa (us)	Keterangan
1	1		Rata Air (Horizontal)
2	4		Jepit Membuka Maks
3	4		Jepit Menutup Maks

Data dan Analisis

1. Sajikan data antara lebar pulsa dan sudut servo dalam **grafik** untuk setiap servo.
2. Carilah hubungan antara lebar pulsa yang diberikan dengan sudut servo yang dihasilkan.

Kesimpulan

Berdasarkan hasil analisis, buatlah kesimpulan yang berkaitan dengan percobaan yang telah anda lakukan.

Tugas

1. Ulangi percobaan diatas dengan menggunakan *command mode*
2. Jelaskan mengenai perintah yang dipakai pada *command mode*

Daftar Pustaka

User Manual SSC-32 Ver 2.0

LAMPIRAN :

Command Formatting for the SSC-32

With the exception of MiniSSC-II mode, all SSC-32 commands must end with a carriage return character (ASCII 13). Multiple commands of the same type can be issued simultaneously in a . All of the commands in a command group will be executed after the final carriage return is received. Commands of different types cannot be mixed in the same command group. In addition, numeric arguments to all SSC-32 commands must be ASCII strings of decimal numbers, e.g. "1234". Some commands accept negative numbers, e.g. "-5678". Programming examples will be provided. ASCII format is not case sensitive. Use as many bytes as required. Spaces, tabs, and line feeds are ignored.

Servo Move or Group Move:

<ch> P <pw> S <spd>... # <ch> P <pw> S <spd> T <time><cr>

<ch> =Channel number in decimal, 0 - 31.
<pw> =Pulse width in microseconds, 500 - 2500.
<spd> =Movement speed in uS per second for one channel. (Optional)
<time> =Time in mS for the entire move, affects all channels, 65535 max. (Optional)
<cr> =Carriage return character, ASCII 13. (Required to initiate action)
<esc> =Cancel the current command, ASCII 27.

Servo Move Example: "#5 P1600 S750 <cr>"

The example will move the servo on channel 5 to position 1600. It will move from its current position at a rate of 750uS per second until it reaches its commanded destination. For a better understanding of the speed argument consider that 1000uS of travel will result in around 90° of rotation. A speed value of 100uS per second means the servo will take 10 seconds to move 90°. Alternately a speed value of 2000uS per second equates to 500mS (half a second) to move 90°.

Servo Move Example: "#5 P1600 T1000 <cr>"

The example will move servo 5 to position 1600. It will take 1 second to complete the move regardless of how far the servo has to travel to reach the destination.

Servo Group Move Example: "#5 P1600 #10 P750 T2500 <cr>"

The example will move servo 5 to position 1600 and servo 10 to position 750. It will take 2.5 seconds to complete the move, even if one servo has farther to travel than another. The servos will both start and stop moving at the same time. This is a very powerful command. By commanding all of the legs in a walking robot with the Group Move it is easy to synchronize complex gaits. The same synchronized motion can benefit the control of a robotic armas well.

You can combine the speed and time commands if desired. The speed for each servo will be calculated according to the following rules:
1. All channels will start and end the move simultaneously.
2. If a speed is specified for a servo, it will not move any faster than the speed specified (but it might move slower if the time command requires).
3. If a time is specified for the move, then the move will take at least the amount of time specified (but might take longer if the speed command requires).

Servo Move Example: "#5 P1600 #17 P750 S500 #2 P2250 T2000 <cr>"

The example provides 1600uS on ch5, 750uS on ch17, and 2250uS on ch2. The entire move will take at least 2 seconds, but ch17 will not move faster than 500uS per second. The actual time for the move will depend on the initial pulse width for ch17. Suppose ch17 starts at position 2000. Then it has to move 1250uS. Since it is limited to 500uS per second, it will require at least 2.5 seconds, so the entire move will take 2.5 seconds. On the other hand, if ch17 starts at position 1000, it only needs to move 250uS, which it can do in 0.5 seconds, so the entire move will take 2 seconds.

Important! Don't issue a speed or time command to the servo controller as the first instruction. It will assume it needs to start at 500uS and will zip there as quickly as possible. The first positioning command should be a normal "# <ch> P <pw>"command. Because the controller doesn't know where the servo is positioned on power up it has to be this way.

Pulse Offset:

<ch>PO <offset value> … # <ch> PO <offset value> <cr>

<ch>=Channel number in decimal, 0 - 31.
<offset value> = 100 to -100 in uSeconds.
<cr>=Carriage return character, ASCII 13.

The servo channel will be offset by the amount indicated in offset value. This makes it easy to setup legs in a robot that do not allow mechanical calibration.

Discrete Output :

<ch><lvl> ... # <ch> <lvl><cr>

<ch>=Channel number in decimal, 0 - 31.
<lvl>=Logic level for the channel, either 'H' for High or 'L' for Low.
<cr>=Carriage return character, ASCII 13.

The channel will go to the level indicated within 20mS of receiving the carriage return.

Discrete Output Example: "#3H #4L <cr>"

This example will output a High (+5v) on channel 3 and a Low (0v) on channel 4.

Byte Output:

<bank> : <value><cr>

<bank>=(0 = Pins 0-7, 1 = Pins 8-15, 2 = Pins 16-23, 3 = Pins 24-31.)
<value>=Decimal value to output to the selected bank (0-255). Bit 0 = LSB of bank.

This command allows 8 bits of binary data to be written at once. All pins of the bank are updated simultaneously. The banks will be updated within 20mS of receiving the CR.

Bank Output Example: "#3:123 <cr>"

This example will output the value 123 (decimal) to bank 3. 123 (dec) = 01111011 (bin), and bank 3 is pins 24-31. So this command will output a "0" to pins 26 and 31, and will output a "1" to all other pins.

Query Movement Status:

Q <cr>

This will return a "." if the previous move is complete, or a "+" if it is still in progress.
There will be a delay of 50uS to 5mS before the response is sent.

Query Pulse Width:

QP <arg><cr>

This will return a single byte (in binary format) indicating the pulse width of the selected servo with a resolution of 10uS. For example, if the pulse width is 1500uS, the returned byte would be 150 (binary).

Multiple servos may be queried in the same command. The return value will be one byte per servo. There will be a delay of least 50uS to 5mS before the response is sent. Typically the response will be started within 100uS.

Read Digital Inputs:
A B C D AL BL CL DL <cr>

A, B, C, or D reads the value on the input as a binary value. It returns ASCII "0" if the input is a low (0v) or an ASCII "1" if the input is a high (+5v).

AL, BL, CL, or DL returns the value on the input as an ASCII "0" if the input is a low (0v) or if it has been low since the last *L command. It returns a high (+5v) if the input is a high and never went low since the last *L command. Simply stated it will return a low if the input ever goes low. Reading the status automatically resets the latch.

The ABCD inputs have a weak pullup (~50k) that is enabled when used as inputs. They are checked approximately every 1mS, and are debounced for approximately 15mS. The logic value for the read commands will not be changed until the input has been at the new logic level continuously for 15mS. The Read Digital Input Commands can be grouped in a single read, up to 8 values per read. They will return a string with one character per input with no spaces.

Read Digital Input Example: "A B C DL <cr>"

This example returns 4 characters with the values of A, B, C, and D-Latch. If A=0, B=1, C=1, and DL=0, the return value will be "0110".

Read Analog Inputs:

VA VB VC VD <cr>

VA, VB, VC, VD reads the value on the input as analog. It returns a single byte with the 8- bit (binary) value for the voltage on the pin.

When the ABCD inputs are used as analog inputs the internal pullup is disabled. The inputs are digitally filtered to reduce the effect of noise. The filtered values will settle to their final values within 8mS of a change. A return value of 0 represents 0vdc. A return value of 255 represents +4.98vdc. To convert the return value to a voltage multiply by 5/256. At power-up the ABCD inputs are configured for digital input with pullup. The first time a V* command is used, the pin will be converted to analog without pullup. The result of this first read will not return valid data.

Read Analog Input Example: "VA VB <cr>"

This example will return 2 bytes with the analog values of A and B. For example if the voltage on Pin A is 2vdc and Pin B is 3.5vdc, the return value will be the bytes 102 (binary) and 179 (binary).

12 Servo Hexapod Sequencer Commands :

LH <arg>, LM<arg>, LL <arg>

Set the value for the vertical servos on the left side of the hexapod. LH sets the high value, i.e. the pulse width to raise the leg to its maximumheight; LM sets the mid value; and LL sets the low value. The valid range for the arguments is 500 to 2500uS.

RH <arg>, RM<arg>, RL <arg>

Set the value for the vertical servos on the right side of the hexapod. RH sets the high value, i.e. the pulse width to raise the leg to its maximumheight; RM sets the mid value; and RL sets the low value. The valid range for the arguments is 500 to 2500uS.

VS <arg>

Sets the speed for movement of vertical servos. All vertical servo moves use this speed. Valid range is 0 to 65535uS/Sec.

LF <arg>, LR <arg>

Set the value for the horizontal servos on the left side of the robot. LF sets the front value, i.e. the pulse width to move the leg to the maximum forward position; LR sets the rear value. The valid range for the arguments is 500 to 2500uS.
RF <arg>, RR <arg>

Set the value for the horizontal servos on the right side of the robot. RF sets the front value, i.e. the pulse width to move the leg to the maximum forward position; RR sets the rear value. The valid range for the arguments is 500 to 2500uS.

HT <arg>

Sets the time to move between horizontal front and rear positions. The valid range for the argument is 1 to 65535uS.

XL <arg>, XR <arg>

Set the travel percentage for left and right legs. The valid range is -100% to 100%. Negative values cause the legs on the side to move in reverse. With a value of 100%, the legs will move between the front and rear positions. Lower values cause the travel to be proportionally less, but always centered. The speed for horizontal moves is adjusted based on the XL and XR commands, so the move time remains the same.

XS <arg>

Set the horizontal speed percentage for all legs. The valid range is 0% to 200%. With a value of 100%, the horizontal travel time will be the value programmed using the HT command. Higher values proportionally reduce the travel time; lower values increase it. A value of 0 will stop the robot in place. The hex sequencer will not be started until the XS command is received.

XSTOP

Stop the hex sequencer. Return all servos to normal operation.
Notes on Hex Sequencer:

1) The following servo channels are used for the Hex Sequencer

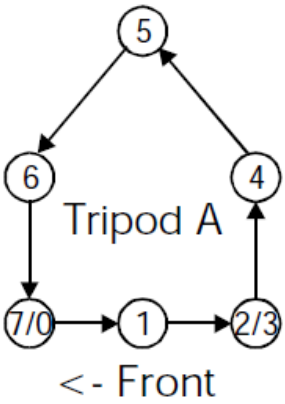
0 = Right Rear Vertical	16 = Left Rear Vertical
1 = Right Rear Horizontal	17 = Left Rear Horizontal
2 = Right Center Vertical	18 = Left Center Vertical
3 = Right Center Horizontal	19 = Left Center Horizontal
4 = Right Front Vertical	20 = Left Front Vertical
5 = Right Front Horizontal	21 = Left Front Horizontal

2) The Hexapod walking gait is an alternating tripod. The tripods are labeled Tripod A and Tripod B. Tripod A consists of {Left Front, Left Rear, Right Center}, and Tripod B consists of {Left Center, Right Front, Right Rear}.

3) While walking, the legs pass through 6 points: (Low Front), (Low Center), (Low Rear), (Mid Rear), (High Center), and (Mid Front). “Center” refers to the mid-point between the Front and Rear pulse widths.

4) The walking sequence consists of 8 states, numbered 0 –7. They are defined below:

State	Tripod A		Tripod B	
	Vertical	Horizontal	Vertical	Horizontal
0	Low	Front to Center	Mid to High	Rear to Center
1	Low	Center to Rear	High to Mid	Center to Front
2	Low	Rear	Mid to Low	Front
3	Low to Mid	Rear	Low	Front
4	Mid to High	Rear to Center	Low	Front to Center
5	High to Mid	Center to Front	Low	Center to Rear
6	Mid to Low	Front	Low	Rear
7	Low	Front	Low to Mid	Rear



In this table, "Front" and "Rear" are modified by the XL and XR commands. A value of 100% results in the movement in the table. Between 0 and 100%, the Front/Rear positions are moved closer to Center. For negative values, Front and Rear are exchanged. For example, with an XL of -100%, in State 0, Tripod A on the left side would be moving Rear to Center, and Tripod B would be moving Front to Center.

5) When a horizontal servo is moving, its speed will be adjusted based on the Front/Rear pulse widths, the XL/XR percentage, and the XS percentage. Regardless of the travel distance from front to rear (adjusted by XL/XR), the total move time will be the HT divided by the XS percentage.

6) When a vertical servo is moving from Low to Mid or from Mid to Low, it will move at the speed specified by the VS command. When a vertical servo is moving from Mid to High or High to Mid, the vertical speed will be adjusted so that the horizontal and vertical movements end at the same time.
7) Any of the Hex Sequencer commands can be issued while the sequencer is operating. They will take effect immediately.

Hex Sequencer Examples:

"LH1000 LM1400 LL1800 RH2000 RM1600 RL1200 VS3000 <cr>"

Sets the vertical servo parameters.

"LF1700 LR1300 RF1300 RR1700 HT1500 <cr>"

Sets the horizontal servo parameters.

"XL50 XR100 XS100 <cr>"

Causes the gradual left turn at 100% speed (and starts the sequencer if it is not already started).
"XL -100 XR 100 XS 50 <cr>"

Causes a left rotate in place at 50% speed.

"XSTOP <cr>"

Stops the sequencer and allows servo channels 0-5, 16-21 to be controlled using the normal servo movement commands.

Query Hex Sequencer State:

XQ <vr>

Returns 1 digit representing the state of the hex sequencer, and the approximate percentage of movement in the state. The high nibble will be '0'to '7', and the low nibble will be '0'to '9'. For example, if the sequencer is 80% of the way through state 5, it will return the value 58 hex.

Get Software Version:

VER <cr>

Returns the software version number as an ASCII string.

Transfer To Boot:

GOBOOT <cr>

Starts the bootloader running for software updates. To exit the bootloader and start running the application, power cycle the control or enter (case sensitive, no spaces).

g0000<cr>

SSC Emulation: Binary format, 3-bytes.
Byte 1: 255, the sync byte
Byte 2: 0 - 31, the servo number
Byte 3: 0 - 250, the pulse width, 0=500uS, 125=1500uS, 250=2500uS